Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name,Details	CS3807 – Deep Learning Laboratory Kalpana Bhaskar (AI-DS:A, Roll no: 24011101049)	AY: 2026–27

Experiment 1
Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T\mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	**Output Range**	**Typical Applications**
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset
Source: UCI Machine Learning Repository
Download Link:
`https://archive.ics.uci.edu/dataset/267/banknote+authentication`

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output
Dataset summary and statistical information.

Task 2: Exploratory Data Analysis
Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation
Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training
Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation
Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Source Code

The complete implementation of the Single Layer Perceptron, including data preprocessing, model training, testing, and result visualization, is provided in the accompanying Kaggle Notebook.
Kaggle Notebook:
`https://www.kaggle.com/code/kalpanabhaskar/single-layered-perceptron`
Github Repository:
`https://github.com/KalpanaBhaskar/Deep-Learning-Laboratory/tree/main/1_SLP`

7. Experimental Results

Training Summary

Dataset Size	4 features x 1732 instances
Train/Test Split	80 : 20 train-test ratio
Learning Rate	0.01 (Doesnt affect misclassified samples as step function is not differentiable)
Epochs	10 (6th Epoch results in least number of misclassified samples)
Final Weights	[-1.0268 -1.2769 -1.1338 -0.0096]
Final Bias	-0.6
Accuracy	0.9964
Precision	0.9919
Recall	1.0000
F1-score	0.9959

Epoch-wise Learning

Epoch	**Errors**	**Weight 1**	**Weight 2**	**Bias**
1	59	-0.6803	-0.9381	-0.3
2	25	-0.7754	-1.0421	-0.4
3	20	-0.8021	-1.1266	-0.4
4	19	-0.8828	-1.1959	-0.5
5	20	-0.9909	-1.2951	-0.5
6	17	-1.0268	-1.2769	-0.6
7	24	-1.0813	-1.3928	-0.6
8	22	-1.1702	-1.3568	-0.6
9	20	-1.1755	-1.4696	-0.6
10	20	-1.3069	-1.5412	-0.6

8. Generated Plots with Interpretation

1. Feature Histograms

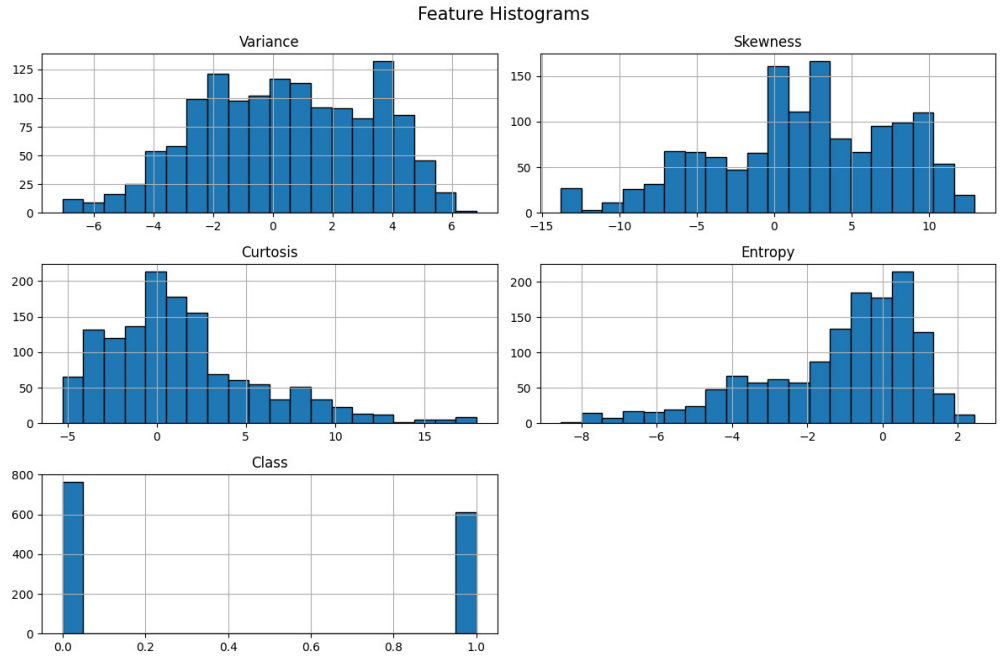

Figure 1: Feature Histograms

- The histogram represents the distribution of the 4 features. Variance and skewness have a wide spread and show multimodality.
- Curtosis is heavily right skewed while Entropy is heavily left skewed.

2. Correlation Heatmap

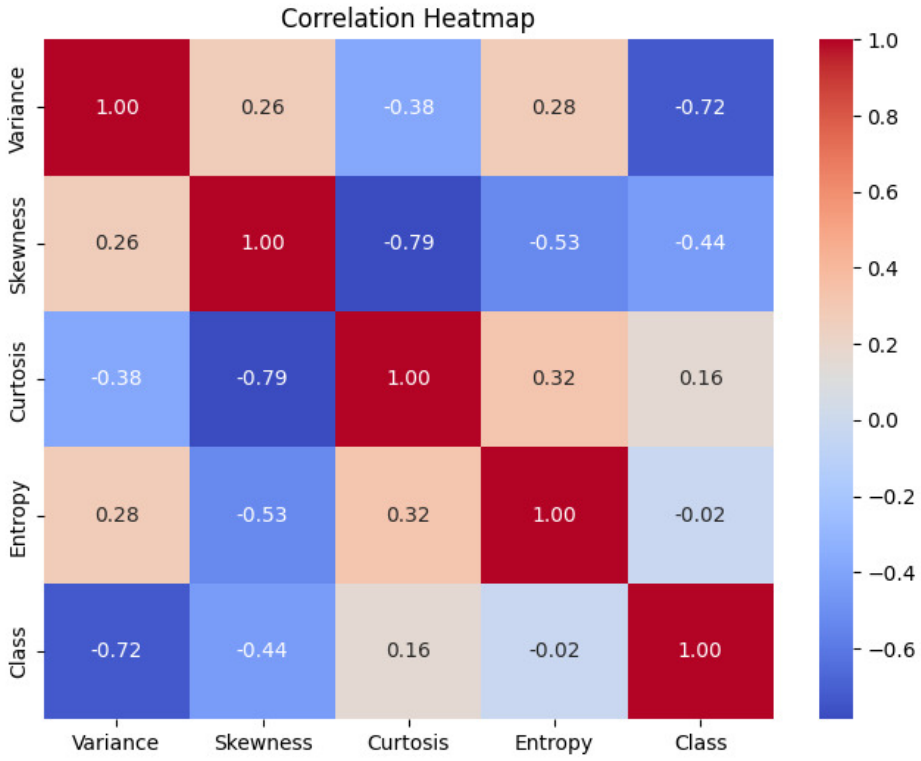

Figure 2: Correlation Heatmap

- The heatmap illustrates the pairwise correlation between all features and the target class. Entropy has almost no linear relationship with the target.
- Variance is the most dominant feature for predicting the target class due to strong negative correlation. As Variance increases, the likelihood of the Class being 1 decreases.

3. Scatter Plot

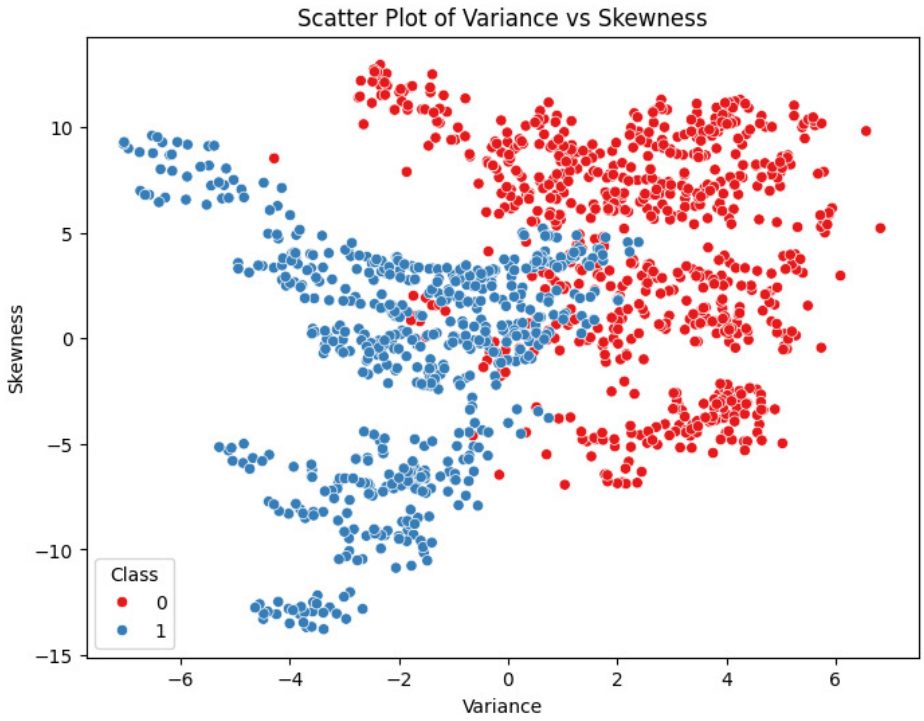

Figure 3: Scatter Plot

- The scatter plot visualizes the distribution of authentic and forged banknotes using two selected features.
- Although a few samples overlap, the classes are reasonably linearly separable.

4. Training Error vs Epoch

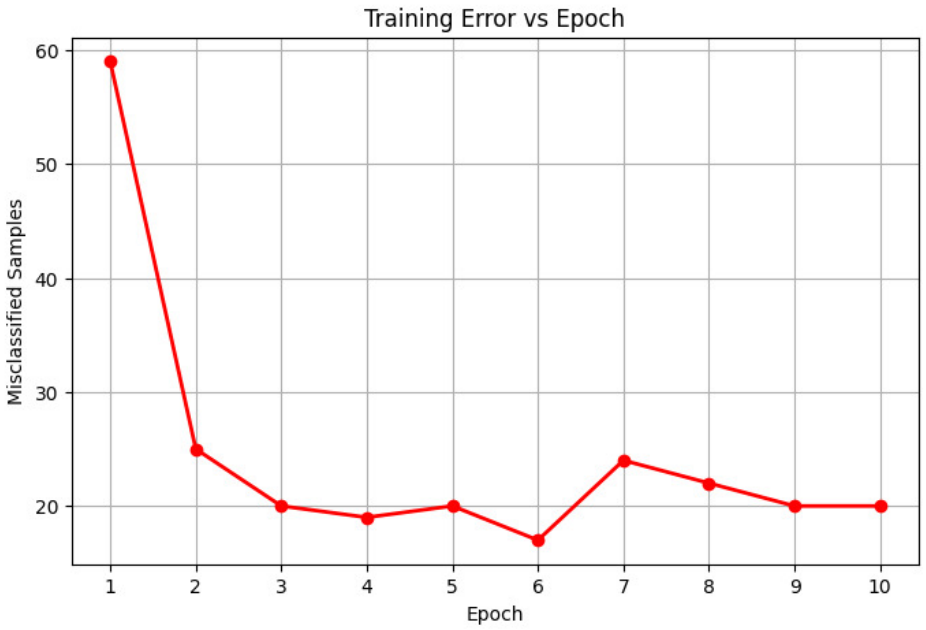

Figure 4: Training Error vs Epoch

- The number of misclassified samples decreases until the 6th epoch, following which it fluctuates between 17 and 22 showing stability.
- The error stabilizes after a few epochs, demonstrating convergence of the learning algorithm.

5. Weight Evolution

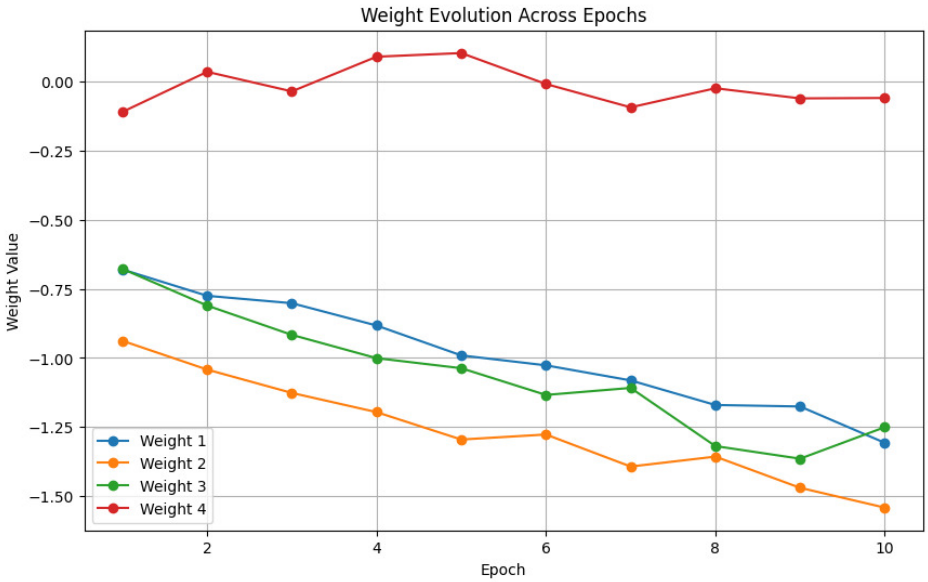

Figure 5: Weight Evolution

- Each weight is updated during training through back-propagation of errors.
- The gradual stabilization of the weights indicates model convergence.

6. Bias Evolution

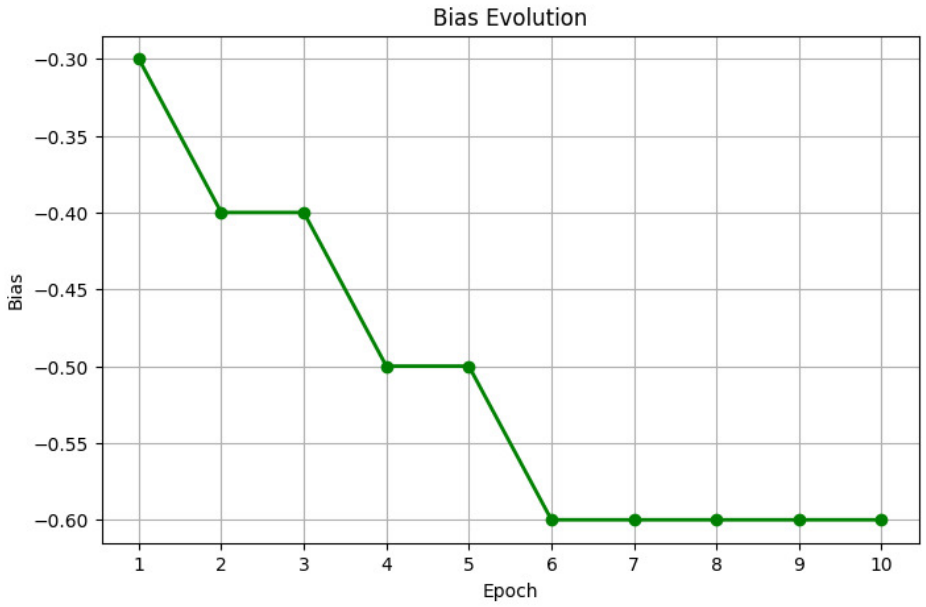

Figure 6: Bias Evolution

- The bias is adjusted throughout training to shift the decision boundary optimally.
- The bias eventually stabilizes, indicating that further updates provide very little improvement in classification.

7. Confusion Matrix

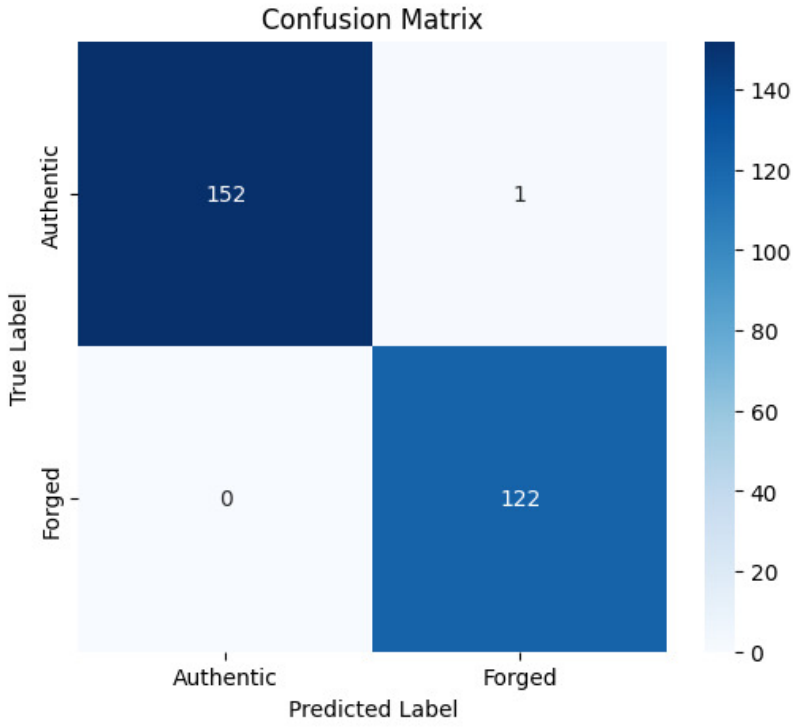

Figure 7: Confusion Matrix

- The confusion matrix summarizes the classification performance based on true positives, true negatives, false positives, and false negatives.
- The model shows no false acceptance of forged items, making it highly reliable. Only 1 false postive proves that the model is robust to

8. Learning Rate Comparison

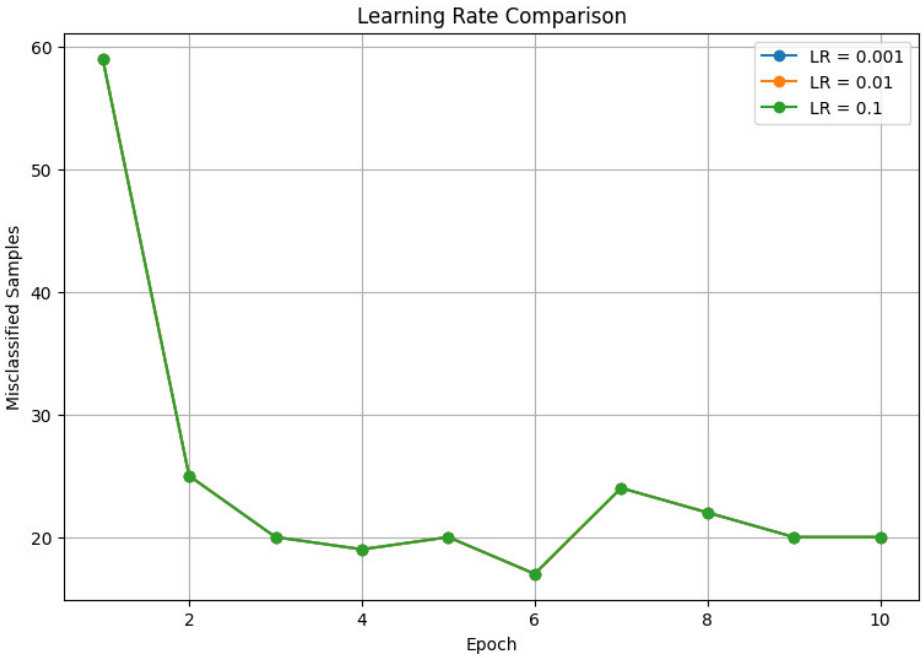

Figure 8: Learning Rate Comparison

- Changing the learning rate only scales the size of the weight vector and not direction as the step function is non differential. Thus during training, the number of misclassified samples dont change with learning rate.

9. Decision Boundary (Optional)

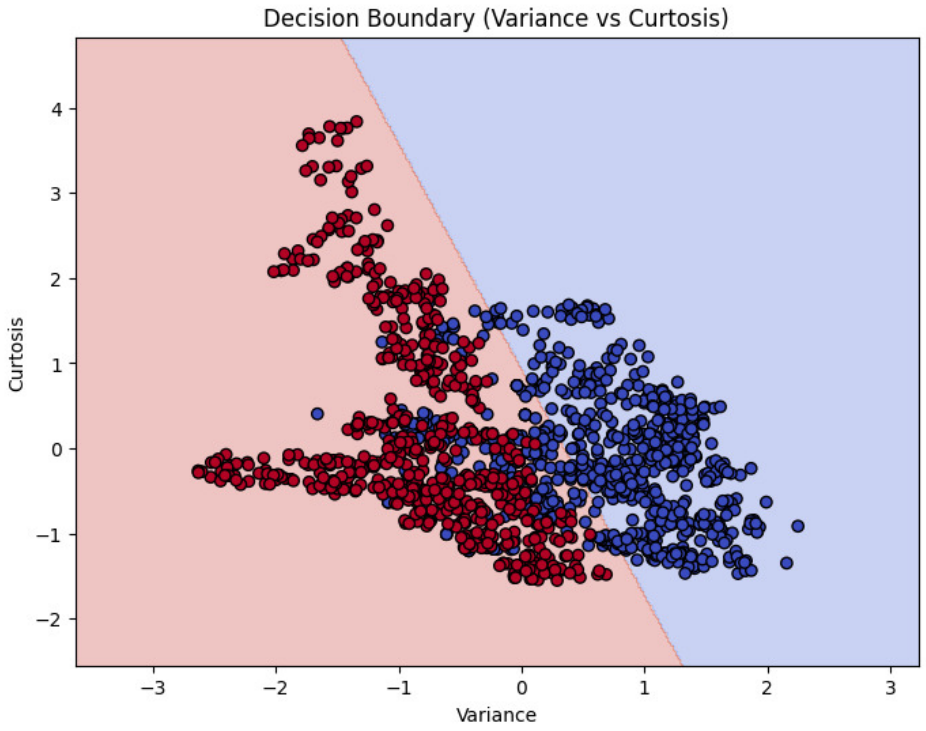

Figure 9: Decision Boundary

- The decision boundary illustrates the linear separation learned by the perceptron using two selected features for visualization.
- Most samples are correctly separated into authentic and forged classes, while overlapping points indicate the limitations of representing a four-dimensional classifier in two dimensions.

9. Discussion

- The Single Layer Perceptron was implemented from scratch using the perceptron learning algorithm, where the model parameters were updated iteratively through weight and bias based on classification errors.
- Exploratory Data Analysis indicated that the Banknote Authentication dataset contains features with different scales and distributions. Feature normalization was applied before training to ensure stable learning and improve convergence.
- Boxplot analysis revealed the presence of outliers, particularly in the **Curtosis** and **Entropy** features. These observations were retained because they represent valid data points rather than measurement errors.
- The number of misclassified samples decreased progressively over successive training epochs until convergence, indicating that the perceptron successfully learned a suitable linear decision boundary.

- Higher learning rates accelerated convergence but could produce larger parameter updates, whereas lower learning rates resulted in a more gradual learning process.
- The confusion matrix provided a detailed evaluation of the classifier by highlighting correctly classified instances as well as the distribution of misclassification errors.
- The decision boundary visualization demonstrated the perceptron's ability to separate the two classes using a linear boundary. The visualization based on two selected features provides only a simplified representation of the model.
- Although the perceptron achieved satisfactory performance on this dataset, its capability is limited to linearly separable problems. More complex non-linear relationships require multilayer neural networks or other advanced machine learning models.

10. Conclusion

- The experiment provided an understanding of the fundamental working principle of an artificial neuron and the Single Layer Perceptron model.
- A perceptron classifier was successfully implemented from scratch using the step activation function and perceptron learning rule.
- Data preprocessing, including feature normalization and train-test splitting, improved the reliability and convergence of the model.
- The model successfully learned from the Banknote Authentication dataset and achieved effective binary classification performance.
- The effect of learning rate on model convergence was analyzed, demonstrating the importance of selecting appropriate hyperparameters.
- The experiment highlighted the advantages and limitations of the perceptron algorithm, forming the foundation for understanding advanced neural networks and deep learning models.
- The study demonstrates that while a Single Layer Perceptron is simple and efficient for linearly separable problems, multilayer neural networks are required for solving complex non-linear classification tasks.

11. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.